My project began by brainstorming various ideas, but I knew I wanted to try to incorporate my Behringer RD-6 drum machine somehow. Building around this constraint, I finally arrived on the idea of using an algorithm based on probability distributions to generate drum patterns for the different sounds on the drum machine.

After landing on this idea of algorithmic drum programming, I explained my proposal to Microsoft Copilot and asked it to help me formulate an outline of steps to implement it. This was an integral step in the process, as it helped me translate my idea into pseudocode and explain the logic of the algorithm in depth before writing the full script.

Before implementing the full algorithm, I first needed to verify that Python could successfully communicate with the drum machine via MIDI. To do this, I tested the compatibility of the Python MIDI package mido with the RD-6. I read through the mido documentation, watched several tutorials, and wrote small test scripts to confirm that my computer recognized the drum machine's MIDI port, and that MIDI messages were being sent to the correct note values. Using the RD-6 manual, I identified the MIDI note mappings for each drum sound and created scripts that first triggered each sound individually and then played a simple one-bar drum pattern.

Once I got everything working with the Python and MIDI interaction, I started working on another small script to properly loop a bar repeatedly. I got this to work but then realized I wanted to try to implement swing into the algorithm, which may mess with the loop timing (which I was correct about). I turned to AI as a troubleshooting consultant for this issue, as I was initially resetting the master clock for every bar, which made the loop seem a bit off due to the swing delay that would then reset when the bar reset. To fix this issue, Copilot suggested that I just maintain a cumulative master clock that can keep track of the swing over the entire period of looping so that it doesn't get reset after every bar, thus cutting off the groove.

After figuring out the logic to implement swing and maintain consistent looping, I started to build out the main program first by defining data structures for context (structures to hold the information of the patterns and define important steps such as downbeats, offbeats, etc.). Then, I defined the sound classes that would allow each sound on the drum machine to have various parameters (such as probabilities, hard rules, and soft preferences) to guide the musical context of the algorithm. From here, I defined the hard rules and soft preferences I wanted to implement into the sound classes and input them where needed. I also defined a function to adjust the probabilities based on these hard rules and soft preferences.

Then, I wrote a function to generate a one bar loop of a drum pattern with the data structures I had defined previously, adding a function to print a grid of the pattern for the user to see. From here, I had to configure MIDI utilities and mapping, implement a playback loop, and define the actions of the program with conditional statements ([s] for save, [n] for next, and [q] to quit). As an additional step, I decided to give the user the ability to specify the BPM and swing amount (from 0.0 to 0.5) from the terminal, so I wrote a function to allow for that choice as well. After inputting information for each sound class and establishing a priority order, the final call in the

program is a call to the session_loop( ) function to start the program's series of choices and probabilities. This call continuously generates drum patterns according to the defined probabilities and rules, sends them to the RD-6 for live playback, and allows the user to interactively explore and save the generated grooves.